BooLUT-CIM: A 52.4 TOPS/W Radix-4 Booth-LUT Digital CIM With Negative-Magnitude-Bits Inversion Storage

Yi Yang, Xiao Tan, Jinwu Chen, Yucheng Du, Tianhui Jiao, Xing Wang, An Guo, and Xin Si

***Abstract*—Compute-in-Memory (CIM) is an effective approach to boost the efficiency of AI hardware by reducing data movement. Digital CIM (DCIM) has garnered extensive attention owing to its immunity to PVT variations and high precision. Recently, a lookup table (LUT)-based DCIM architecture was proposed, which stores the precomputed summations of adjacent weights in memory, thereby alleviating the substantial overhead of adder trees in traditional DCIM designs. However, the efficiency of LUT-based DCIM is difficult to further improve due to the bit-serial input method and the power-hungry LUT readout operations. To address this challenge, this brief presents a BooLUT-CIM design that integrates the advantages of LUT-based CIM and Booth algorithm, reducing the quantity of lookup and addition operations by a half. A LUT compression method derived from computation-friendly numerical relationships is utilized to reduce circuit overhead. Additionally, a negative magnitude-bits inversion storage scheme is proposed to reduce inefficient discharge, saving 38% of LUT array readout power on average. A prototype of this design is implemented in 28nm CMOS technology for validation. Simulation results show that the macro achieves an energy efficiency of 52.4 TOPS/W at 0.9V, 400 MHz, which is 1.3× higher than prior arts.**

***Index Terms*—Compute-in-memory, lookup table, Booth algorithm, sparsity, deep neural networks.**

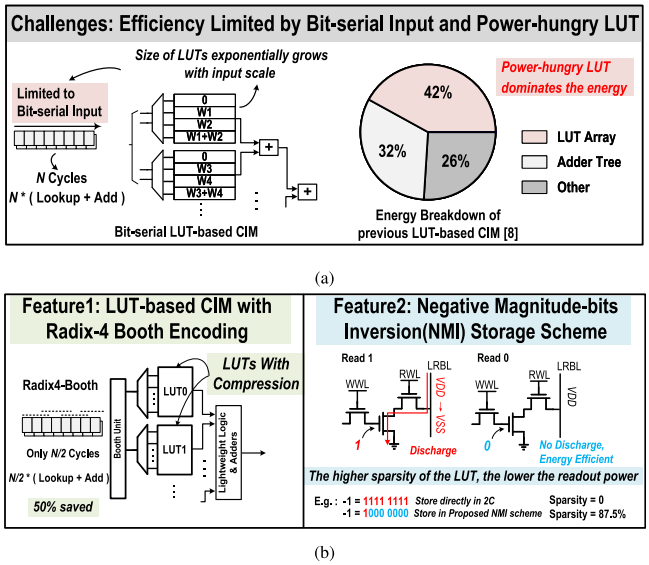


Fig. 1. (a) Challenges of LUT-based DCIM Architecture. (b) Features of proposed BooLUT-CIM.

I. Introduction

THE INCREASING scale and complexity of deep neural networks (DNNs) have exposed notable drawbacks of the traditional Von Neumann architecture. This is predominantly because frequent data movement between memory and processing units creates a crucial bottleneck in system performance. Compute-in-Memory (CIM) has emerged as a promising solution to address this challenge. By integrating computing units into memory arrays, the CIM architecture significantly reduces the energy and latency associated with data movement, achieving high energy efficiency [1], [2], [3].

Digital Computing-in-Memory (DCIM) architectures have been widely studied at macro level and beyond due to their superior precision, robustness, and scalability in advanced process nodes [3], [4], [5], [6], [7], [8], [9]. However, the energy and area efficiency of traditional DCIM designs are constrained by the heavy overhead of the inherent adder trees [8], [10], [11]. To address this bottleneck, recent innovations propose a lookup table (LUT)-based DCIM approach. He et al. proposed an eDRAM LUT-based CIM macro that achieves high compute and memory density by storing precomputed weight sums in customized shrunk 3-transistor (3T) gaincells [12]. Zhu et al. presented an SRAM LUT-based DCIM macro for integer multiply-and-accumulate (MAC) operations, omitting the overhead of the first stage adder tree by storing precomputed summations (PreSums) of adjacent weights [13].

Although LUT-based CIM architecture reduces the power and area overhead of the adder tree, it still faces several challenges, as shown in Fig. 1(a). Firstly, due to the exponential growth nature of LUT size with increasing input scale, previous LUT-based CIM designs have employed a bit-serial approach [12], [13]. This method necessitates N cycles of lookup and addition to complete the MAC computation for N-bit inputs, thereby limiting the energy efficiency and increasing the latency of LUT-based CIM macros. Secondly, power-hungry LUT readout operations dominate the power

Received 4 March 2025; revised 4 July 2025; accepted 16 July 2025. Date of publication 21 July 2025; date of current version 4 November 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2022ZD0118902; in part by the National Natural Science Foundation of China under Grant 62204036 and Grant 92264203; and in part by the Jiangsu Provincial Key Research and Development Program under Grant BE2023020-1. This brief was recommended by Associate Editor A. Liscidini. *(Corresponding author: Xin Si.)*

The authors are with the School of Integrated Circuits, Southeast University, Nanjing 210096, China (e-mail: xinsi@seu.edu.cn).

Digital Object Identifier 10.1109/TCSII.2025.3590745

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

consumption, emerging as the primary bottleneck for overall energy efficiency.

To address these challenges, as illustrated in Fig. 1(b), we propose the BooLUT-CIM design which features two primary innovations:

(1) A LUT-based CIM design compatible with Radix-4 Booth input encoding, reducing the number of lookups and additions by half for each MAC computation. A LUT compression opportunity derived from computation-friendly numerical relationships is utilized to reduce storage redundancy.

(2) A negative magnitude-bits inversion storage scheme to enhance the bit-level sparsity of data stored in LUTs, which helps reduce inefficient discharges during readout. Evaluations based on network data indicate that this method improves readout energy efficiency by 1.59× on average.

For validation, a BooLUT-CIM prototype is implemented in 28nm CMOS technology. Simulation results show that the macro achieves an energy efficiency of 52.4 TOPS/W, indicating a 1.3× improvement over prior arts.

II. Background

A. Radix-4 Booth Algorithm

Digital CIM designs often employ a bit-serial input method [2], [14]. By breaking down the multiplication into a series of bitwise operations, multiplier circuitry overhead is significantly reduced. As illustrated in (1), an n-bit bit-serial input IN_{bs} is fed into the CIM array for bit-by-bit multiplication with the weight W and addition, followed by weighted summation in the accumulator. However, this process requires n clock cycles to complete the computation.

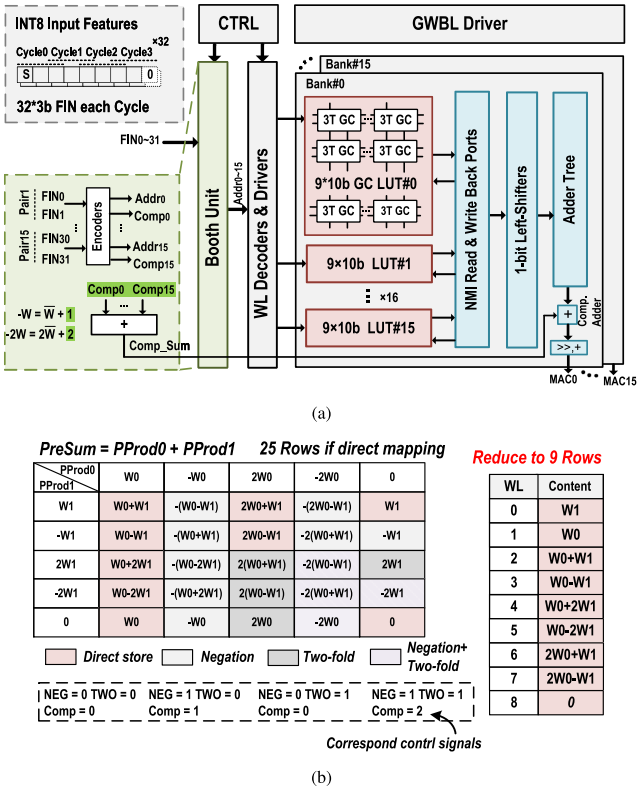
$$IN_{\text{bs}} \times W = \sum_{i=0}^{n-1} \left(IN_{\text{bs}}[i] \times W \times 2^i \right) \tag{1}$$

$$IN \times W = \sum_{i=0}^{\lfloor \frac{n-1}{2} \rfloor} \left(-2IN[2i+1] + IN[2i] + IN[2i-1] \right) \times W \times 4^i \tag{2}$$

Booth algorithm is an efficient technique for reducing redundant calculations in multiplication by encoding the input bits. Among the various versions of the Booth algorithm, the Radix-4 Booth algorithm stands out for its efficiency and effectiveness in the design of AI accelerators and CIMs [5], [15]. Compared to the Radix-2 algorithm, the Radix-4 version reduces the number of partial products (PProds) generated during the multiplication process by processing two new bits per cycle [16]. Compared to higher-order versions such as Radix-8, Radix-4 offers a better balance between encoding and computational complexity. As demonstrated in (2), where $IN[-1] = 0$ and $IN[n] = IN[n-1]$ providing boundary padding, and the operator $\lfloor \cdot \rfloor$ representing the floor function, the Radix-4 encoding scheme achieves approximately 50% reduction in the number of bitwise operations. The partial products of Radix-4 Booth encoding are shown in Table I.

TABLE I
PARTIAL PRODUCTS OF RADIX-4 BOOTH ENCODING

$IN[2i+1:2i-1]$	*Partial Product*
000/111	0
001/010	W
101/110	-W
011	2W
100	-2W



(a)

(b)

Fig. 2. (a) Architecture of the proposed CIM macro. (b) A LUT compression method leveraging computation-friendly numerical relationships.

III. Proposed BooLUT-CIM Macro

A. Architecture of the Proposed CIM Macro

The block diagram of the BooLUT-CIM macro architecture is depicted in Fig. 2(a). It comprises a 144×160-bit 3T gaincell LUT array, a Booth Unit, Wordline (WL) decoders and drivers, and other essential peripheral circuits and control modules.

The LUT array is partitioned into 16 banks. Each bank consists of 16 LUTs, with each LUT paired with one Negative Magnitude-bits Inversion (NMI) read and write-back port and one 1-bit left-shifter. The 16 LUTs within each bank share an adder tree for accumulation and a compensation adder for correction of negation operations. The computed result is then forwarded to the shift-and-add unit to generate the final multiply-and-accumulate result.

32 channels of 3-bit input features $\text{FIN}_{0\sim32}$ are fed into the Booth Unit each cycle, where two adjacent signals are paired for a single LUT. The Booth Unit generates address signals for LUTs and control signals for NMI read ports and left-shifters based on booth algorithm. As marked in Fig. 2(a), the mathematical relationships $\overline{W}+1 = -W$ and $2\overline{W}+2 = -2W$ require compensating each negation operation with an additional 1 or 2. These compensation values $\text{Comp}_{0\sim15}$ are

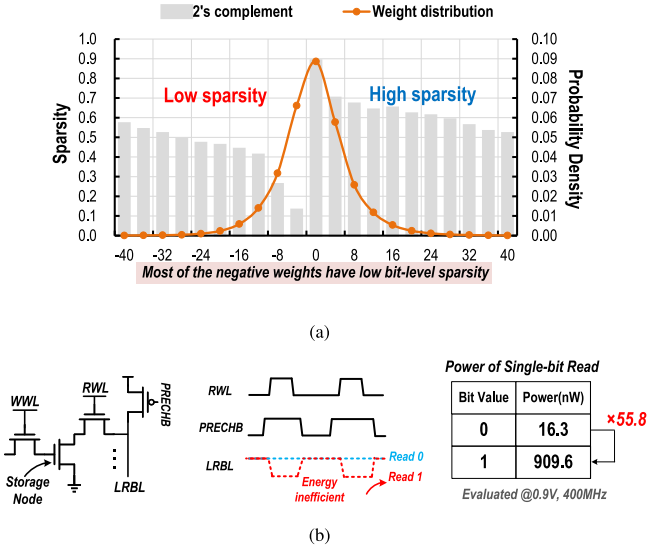


Fig. 3. (a) Distribution of weights and bit-level sparsity statistics. (b) Power analysis of read operations.

aggregated within the Booth unit and sent as Comp_sum to the compensation adder for correction.

The LUT compression details are illustrated in Fig. 2(b). Owing to Radix-4 Booth encoding, the possible values of the PProds in each cycle are confined to one value of $\{-2W, -W, 0, W, 2W\}$. Consequently, directly mapping the sum of adjacent PProds to the LUT yields 25 potential combinations, which incurs a substantial overhead 25-row LUTs as shown in the left panel. However, upon closer examination, it is observed that these 25 combinations exhibit specific numerical relationships: a two-fold relationship and a negation relationship. Both of these relationships are computation-friendly and incur minimal overhead in circuit implementation compared with additions. Therefore, by introducing a 1-bit shifter in conjunction with reconfiguring the read-out circuitry to a negation circuit, we successfully compressed the LUT size to 9 rows as shown in the right panel with minimal circuit overhead.

B. Negative Magnitude-Bits Inversion Storage Scheme

Fig. 3(a) illustrates the weight distribution and bit-level sparsity in deep neural networks, showing that most weight values are concentrated near zero. This distribution leads to high bit-level sparsity for positive weights, while negative weights exhibit low sparsity due to the influence of the sign bit in 2's complement (2C) representation.

As shown in Fig. 3(b), 3T gaincell stores data bit in the form of charge at the storage node. When storing "1", the establishment of a discharge path causes RBL to discharge during each read operation. This discharge operation necessitates a full swing recharge of the RBL in the subsequent cycle. In contrast, reading "0" does not create a discharge path, making it more energy-efficient. Simulation results show that reading '1' consumes 55.8 times more power than reading '0'. Therefore, increasing the bit-level sparsity within the LUT can significantly reduce LUT readout power.

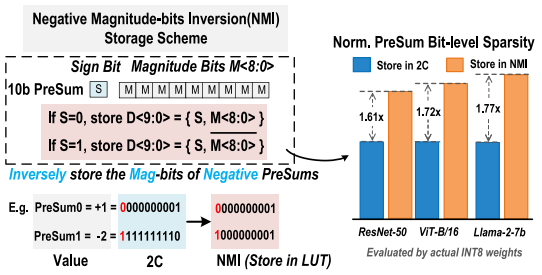


Fig. 4. Negative Magnitude-bits Inversion storage scheme.

Based on these observations, we propose the Negative Magnitude-bits Inversion Storage Scheme to increase bit-level sparsity in LUTs, as shown in Fig. 4. Given that the majority of negative weights exhibit low sparsity, we inversely store the magnitude bits of negative values in the LUT to enhance sparsity. Additionally, it can be effortlessly converted back to 2's complement for subsequent operations through customized readout ports. Statistical results demonstrate that the bit-level sparsity of PreSums stored in NMI is increased by more than 1.61×, while the readout energy efficiency of the LUT is improved by more than 1.56× due to this sparsity benefit.

C. Design Details of LUT Array

Fig. 5 presents the design details of the Booth LUT Array, which supports three working modes: CIM, Refresh, and Encode. In CIM mode, the array performs LUT-based computation and subsequent calculations. In Refresh mode, local refresh operations are executed. In Encode mode, adjacent LUTs utilize the first-stage adder for local encoding.

The left panel illustrates the circuit structure of the LUT, which is composed of a 9×10 array of 3T gaincells. The read and write operations of the gaincells are controlled by RWLs and WWLs respectively. Data is read out via LRBLs, and written back through LWBLs when refreshing and local encoding. GWBLs are used for global write.

The center panel illustrates the design details of the NMI read and write-back ports and the data paths in different modes. In CIM and Encode modes, the NMI read port converts data stored in NMI format to 2's complement by selecting magnitude bits PreSum⟨0:8⟩ based on the sign bit PreSum⟨9⟩. Similarly, the NMI read port performs negation operations by selecting the sign bit PreSum⟨9⟩ using the NEG signal generated by the Booth Unit. In refresh mode, signals on LRBLs are directly written back to LWBL via inverters.

The right panel illustrates the subsequent processes in CIM and Encode modes. In CIM mode, PreSum is shifted according to the TWO signal and then fed into the adder tree to complete the computation. In Encode mode, the output of the first-level adder, after converting from 2's complement to NMI format, is written back to the designated row to complete local encoding. Notably, the least significant bit of the first-level adder is extended to a full adder with a Sub signal connected to the carry input, enabling it to be reconfigured as a subtractor during encoding.

Fig. 6 presents the detailed local PreSum encoding procedure in detail. The procedure consists of two main phases:

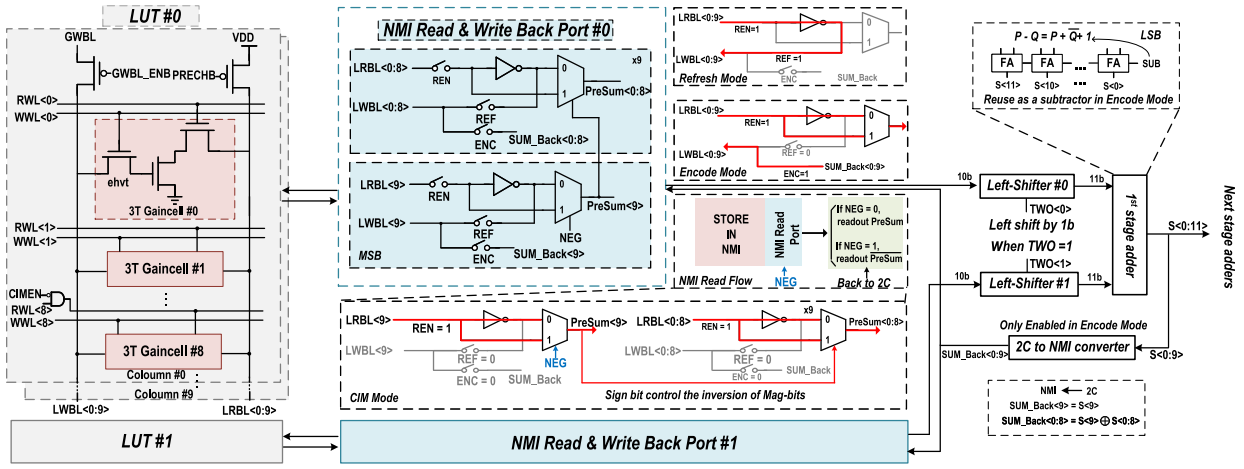


Fig. 5. Design of the LUT array and details of NMI read and write back port.

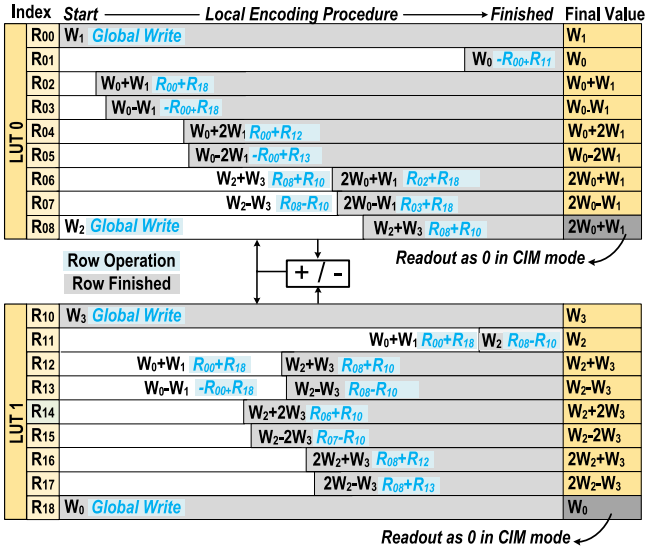


Fig. 6. Procedure of local encoding.

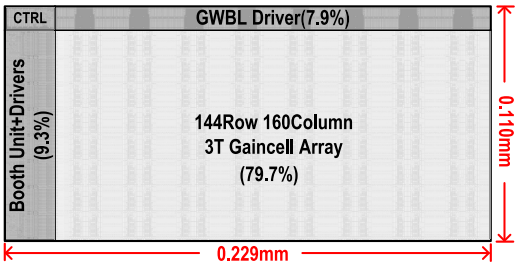


Fig. 7. Layout of the proposed BooLUT-CIM macro.

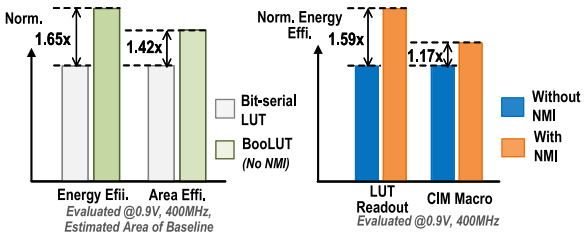


Fig. 8. Analysis of the proposed BooLUT and NMI storage scheme.

First, four initial weights are programmed into adjacent LUTs through global write operations. Then, this LUT pair utilizes the first-level adder to perform addition and subtraction operations, with the results being written back to complete the PreSum encoding. The encoding process for all LUTs can be executed in parallel, enhancing efficiency. Additionally, in CIM mode, RWL⟨8⟩ is gated, forcing its readout to zero for computational purposes. This approach eliminates the latency and power consumption associated with external read and write operations in global encoding.

IV. Implementation and Evaluation

A. Design Implementation and Simulation

A 22.5Kb prototype was implemented using 28nm technology. This macro delivers 16 channels of MAC results within 4 cycles, achieving a throughput of 102 GOPS at 400 MHz. As shown in Fig. 7, the macro occupies an area of 0.229mm × 0.110mm, corresponding to an area efficiency of 4.05 TOPS/mm^2.

Experiment results demonstrate an energy efficiency of 52.4 TOPS/W for INT8 MAC computations at 0.9V and 400 MHz. The sparsity of input activations and weight data were set to 50%. The eDRAM array's refresh interval is configured to 250 ns to ensure robust cell retention, providing a 1.5× margin relative to the worst-case retention time at fast-fast corner.

B. Analysis and Comparison

Fig. 8 clearly illustrates the performance improvements achieved by the proposed BooLUT and NMI storage schemes. The left panel compares the performance of the BooLUT scheme with a baseline implementation based on a bit-serial LUT. It can be observed that BooLUT enhances energy efficiency by 1.65× and improves the area efficiency by 1.42×. The right panel presents the power optimization attained through the proposed NMI storage scheme. Specifically, the implementation of the NMI scheme increases the energy

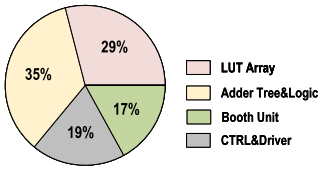


Fig. 9. Power breakdown of the proposed BooLUT-CIM macro.

TABLE II
COMPARISON WITH PREVIOUS WORKS

Metric	JSSC'23 [5]	ISSCC'24 [12]	TCASII'24 [13]	TCASII'23 [6]	This work
Technology	28nm	28nm[a]	28nm	40nm	28nm
Cell	SRAM	3T Gaincell	SRAM	SRAM	3T Gaincell
Array Size	12Kb	40Kb	16Kb	34Kb	22.5Kb
Area (mm^2)	6.69	0.017	0.038	0.061	0.025
Data Type	INT8, BF16	INT8	INT8	INT1-16	INT8
Full Precision?	Yes	Yes	No	Yes	Yes
LUT-Based?	No	Yes	Yes	No	Yes
Area Efficiency (TOPS/mm^2)	0.201	16.2	0.68	0.67[b]	4.05[c]
Energy Efficiency (TOPS/W)	36.5 @INT8	18.1	40.6	10.18[b] @INT8	52.4[c]

[a]28nm HKMG Technology with shrinked 3T gaincells.
[b]Normalized to 28nm technology node.
[c]Simulation results @0.9V, 400 MHz.

efficiency of the LUT Array by 1.59× and improves the overall energy efficiency of the CIM macro by 1.17×. The power breakdown of the macro, as depicted in Fig. 9, reveals a significant decrease in the proportion of power consumed by the LUT array. This reduction is primarily attributed to the employed storage optimization techniques.

Detailed performance metrics of the macro and its comparison with state-of-the-art works are provided in Table II. Our design achieves a 1.3× improvement in energy efficiency over prior works while maintaining full-precision capabilities.

V. CONCLUSION

This brief presents a novel BooLUT-CIM design. By incorporating the Radix-4 Booth algorithm into the LUT-based CIM architecture and leveraging computation-friendly numerical relationships, the computation workload of the array and cycles required for MAC operations is reduced by approximately 50%. A negative magnitude-bits inversion storage scheme is proposed to lower the power consumption of LUTs. A 22.5Kb prototype is implemented in 28-nm CMOS technology. revisionSimulation results demonstrate that the proposed BooLUT-CIM macro achieves an energy efficiency of 52.4 TOPS/W, representing a 1.3× improvement over prior arts.

REFERENCES

[1] X. Si et al., "A twin-8T SRAM computation-in-memory unit-macro for multibit CNN-based AI edge processors," *IEEE J. Solid-State Circuits*, vol. 55, no. 1, pp. 189–202, Jan. 2020.
[2] H. Kim, T. Yoo, T. T.-H. Kim, and B. Kim, "A reconfigurable SRAM-based digital bit-serial compute-in-memory macro for processing neural networks," *IEEE J. Solid-State Circuits*, vol. 56, no. 7, pp. 2221–2233, Jul. 2021.
[3] J.-W. Su et al., "A 28 nm 64 Kb inference-training two-way transpose multibit 6T SRAM compute-in-memory macro for AI edge chips," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2020, pp. 240–242.
[4] A. Guo et al., "A 28nm 64-kb 31.6-TFLOPS/W digital-domain floating-point-computing-unit and double-bit 6T-SRAM computing-in-memory macro for floating-point CNNs," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2023, pp. 128–130.
[5] F. Tu et al., "A reconfigurable digital computing-in-memory processor with unified FP/INT pipeline for cloud AI acceleration," *IEEE J. Solid-State Circuits*, vol. 58, no. 1, pp. 243–255, Jan. 2023.
[6] C. Zhang, M. Wang, Y. Mai, C. Tang, and Z. Yu, "A high-density and reconfigurable SRAM-based digital compute-in-memory macro for low-power AI chips," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 70, no. 9, pp. 3589–3593, Sep. 2023.
[7] W. Jiang, C. Caron, P. Avasare, M. Pauwels, M. Verhelst, and W. Dehaene, "HUNBN, a 1.77MB digital in-memory-compute SoC for edge applications achieving 126 TOPs/W (4b) at macro level and 24 TOPs/W at SoC level," in *Proc. IEEE Eur. Solid-State Electron. Res. Conf. (ESSERC)*, Sep. 2024, pp. 137–140, doi: 10.1109/ESSERC62670.2024.10719477.
[8] W.-S. Khwa et al., "34.2 a 16nm 96Kb integer/floating-point dual-mode-gain-cell-computing-in-memory macro achieving 73.3-163.3TOPS/W and 33.2-91.2TFLOPS/W for AI-Edge devices," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, Feb. 2024, pp. 568–570, doi: 10.1109/ISSCC49657.2024.10454447.
[9] M. Lou et al., "Area-efficient and low-power 8T compute-SRAM bitcell design for digital compute-in-memory macros in 22nm CMOS," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, early access, Jun. 25, 2025, doi: 10.1109/TCSII.2025.3583043.
[10] T.-H. Wen et al., "34.8 a 22nm 16Mb floating-point ReRAM compute-in-memory macro with 31.2TFLOPS/W for AI edge devices," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, Feb. 2024, pp. 580–582, doi: 10.1109/ISSCC49657.2024.10454468.
[11] X. Wang et al., "14.3 a 28nm 17.83-to-62.84TFLOPS/W broadcast-alignment floating-point CIM macro with non-two's-complement MAC for CNNs and transformers," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, Feb. 2025, pp. 254–256, doi: 10.1109/ISSCC49661.2025.10904738.
[12] Y. He et al., "A 28nm 2.4Mb/mm^2 6.9 - 16.3TOPS/mm^2 eDRAM-LUT-based digital-computing-in-memory macro with in-memory encoding and refreshing," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2024, pp. 578–580.
[13] H. Zhu, H. Zhang, S. He, M. Li, X. Zeng, and C. Chen, "A LUT-based compute-in-memory macro with trident read bit-line and partial product pruning," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 71, no. 5, pp. 2544–2548, May 2024.
[14] Y.-D. Chih et al., "An 89TOPS/W and 16.3tops/mm^2 all-digital SRAM-based full-precision compute-in memory macro in 22nm for machine-learning edge applications," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2021, pp. 252–254.
[15] Q. Cheng et al., "A low-power sparse convolutional neural network accelerator with pre-encoding radix-4 Booth multiplier," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 70, no. 6, pp. 2246–2250, Jun. 2023.
[16] X. Cui, W. Liu, X. Chen, E. E. Swartzlander, and F. Lombardi, "A modified partial product generator for redundant binary multipliers," *IEEE Trans. Comput.*, vol. 65, no. 4, pp. 1165–1171, Apr. 2016.